

ECE 437

Final Report

Esharaqa Afreen Jahid and Akram Mohamed

June 21, 2026

1 Executive Overview

This report provides a comprehensive performance analysis comparing four processor architectures: a single-cycle processor, a pipelined processor without a cache hierarchy, a pipelined processor with a cache hierarchy, and a multi-core processor. The single-cycle design prioritizes architectural simplicity by executing each instruction within one clock cycle, though it is limited by a long clock period to accommodate the worst-case critical path. The pipelined architecture aims to increase throughput by overlapping instruction execution, while the addition of a cache hierarchy is intended to mitigate the significant performance penalties associated with high memory latency. Furthermore, the multi-core design introduces hardware-level parallelism and coherence control to evaluate speedup for multi-threaded workloads.

To evaluate these designs, performance data was collected using the `merge-sort` and `dual-mergesort` benchmarks across varying memory latencies of 0, 2, 6, and 10 cycles. We analyze these architectures by comparing maximum frequency, average clocks per instruction (CPI), and total execution time against FPGA resource utilization. We expect the data to validate that while pipelining increases raw frequency, the cache hierarchy is essential for maintaining performance as memory latency scales. Additionally, we anticipate the multi-core design will demonstrate a measurable speedup in parallel execution despite the overhead of coherence protocols. The remainder of this report details the hardware implementation and block diagrams for each design, presents a tabulated summary of experimental results, and concludes with an analysis of the trade-offs between hardware complexity and computational throughput.

In this section, we discuss the architectural implementations of our processors. Figure 1 illustrates the single-cycle design, while Figure 4 shows the five-stage pipelined version.



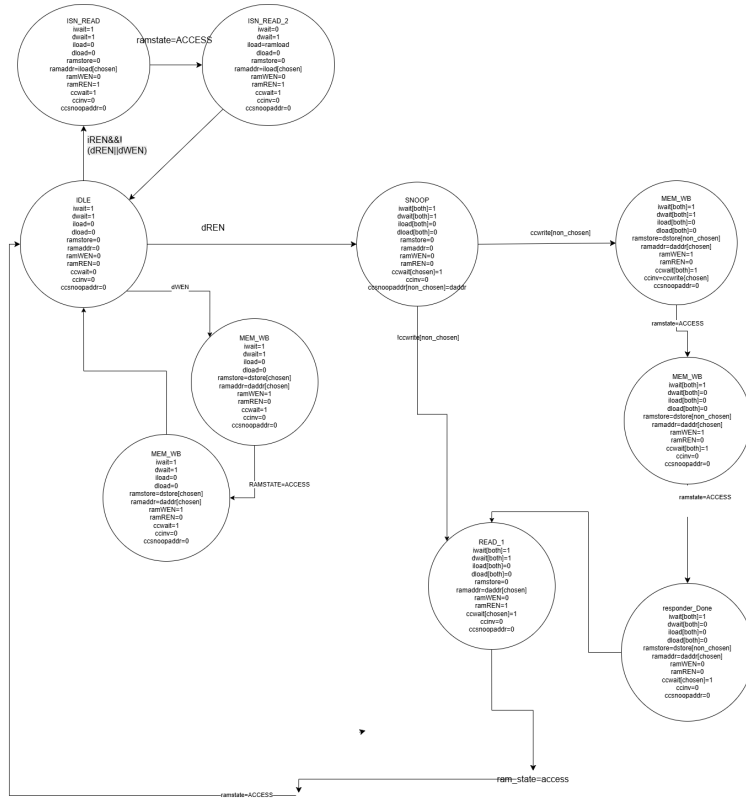


Figure 3: Memory Control Finite State Machine

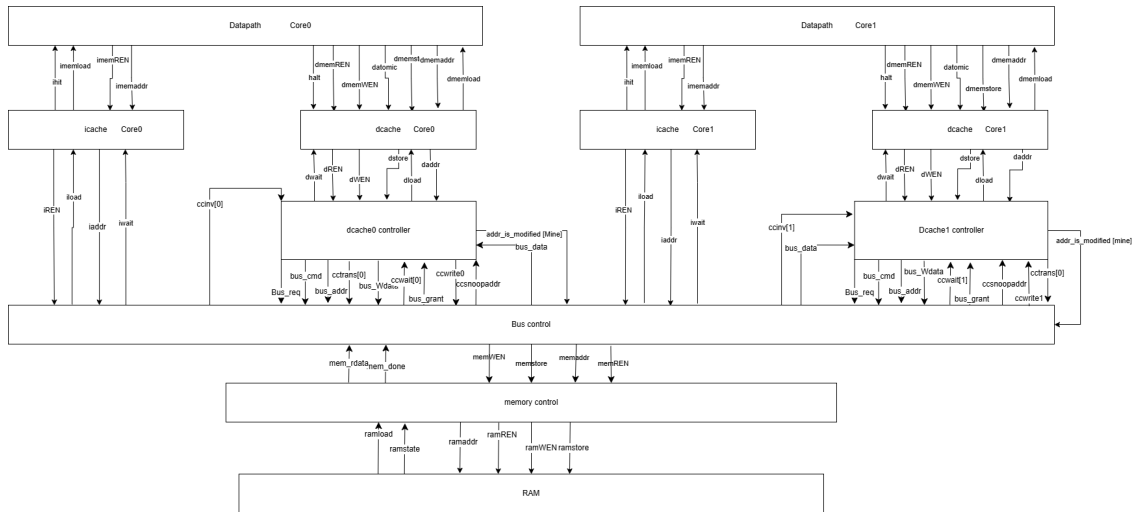


Figure 4: Hierarchical Multicore Diagram

3 Results

For the performance evaluation, we used the mapped version of our designs with gate timing and executed the `merge-sort.asm` (`dual-mergesort.asm` for multicore) test program. Data was collected across memory latencies of $LAT = 0, 2, 6$, and 10. Due to some difficulties collecting exhaustive metrics for the multicore processor across all latencies, We collected the Max Frequency information from the (`test.coherence1.asm`), and the number of cycles from (`dual-mergesort.asm`).

3.1 Performance Formulae

The following formulae were used for all performance calculations:

- Average CPI = $\frac{\text{Total Clock Cycles}}{\text{Total Number of Instructions}}$
- Ideal Latency = $\frac{1}{\text{Max Frequency}}$
- Total Execution Time = Average CPI $\times$ Total Instructions $\times$ Clock Period

3.2 Performance Comparison at Memory Latency = 6

Table 1 outlines the overall performance metrics, hardware resource utilization, and speedup for all four major processor designs at a fixed memory latency of 6 clock cycles.

Metric (LAT = 6)	Single-Cycle	Pipeline (No Cache)	Pipeline (With Cache)	Multicore
Max Frequency (MHz)	40.49	91.28	92.64	77.68
Average CPI	5.11	7.03	2.87	4.84
Total Execution Time (μs)	682.34	416.87	167.66	140.55
Ideal Latency (ns)	24.70	10.96	10.80	12.87
Slice LUTs	1504	1820	3223	7682
Slice Registers	1131	1580	4190	8270
F7 and F8 Muxes	256 F7, 72 F8	273 F7, 64 F8	703 F7, 108 F8	1591 F7, 310 F8
Speedup (vs. Sequential)	–	–	–	1.19x

Table 1: Comprehensive Performance and Resource Utilization Comparison (LAT = 6)

3.3 Execution Time Across Memory Latencies

Table 2 compares the total execution times for all major architectural versions across latencies of 0, 2, 6, and 10.

Processor Design	LAT = 0 (μs)	LAT = 2 (μs)	LAT = 6 (μs)	LAT = 10 (μs)
Single-Cycle	176.11	341.17	682.34	1023.51
Pipeline (No Cache)	108.39	212.16	416.87	621.58
Pipeline (With Cache)	132.24	146.04	167.66	189.86
Multicore	99.79	113.24	140.55	168.85

Table 2: Total Execution Time Across Varying Memory Latencies

Based on the execution times in Table 2, we can estimate the threshold where caches begin to improve performance. At **LAT = 0**, the Pipeline design *without* a cache is actually faster (108.39 μs vs 132.24 μs) because it avoids the inherent cycle delays and overhead introduced by the cache unit. However, at **LAT = 2**, the Pipeline *with* a cache overtakes the baseline pipeline (146.04 μs vs 212.16 μs). Therefore, the RAM latency at which the cache starts heavily benefiting performance is beyond 2 clock cycles. Next at **LAT = 6**, The no-cache pipeline increases to 416.87 μs and 621.58 μs , while the cached pipeline only increases to 167.66 μs and 189.86 μs . which shows the gap increasing. Meanwhile, the multicore design At **LAT = 6** completes execution in 140.55 μs compared to 167.66 μs for the cached single-core pipeline. This shows a speedup of approximately 1.19 as the multicore divides the workload over 2 cores instead of one. However the speedup is not ideal due to the overhead coming from synchronization, memory sharing, ...etc. This shows that the multicore design provides the best overall execution time at the cost of much higher resource utilization.

4 Conclusion

The data indicates that while the pipelined processor achieves a significantly higher clock frequency than the single-cycle design, its performance is heavily influenced by memory latency. As LAT increases, the Average CPI rises due to mandatory stall cycles. The addition of the cache hierarchy provided the largest improvement among the single-core designs. adding cache introduced new control complexity and extra hardware. On the other hand it reduced the impact of memory stalls as RAM latency increased. The results show that caching becomes important when memory access is slower. Finally the multicore design achieved the best overall execution time, especially starting from $LAT = 6$, where it achieved a speedup of approximately 1.19x. This improvement costed the system increased LUTs, registers, and mux usage. The speedup was also not perfect because of synchronization overhead, shared memory behavior, and coherence-related complexity. Based on the collected results, the cached pipelined design provides the best balance between performance and resource efficiency.

5 Contributions

- **Esharaqa Afreen Jahid:** Pipelined datapath implementation, forwarding unit logic, Dcache design, cache integration, parallel algorithms, palgorithm.
- **Akram Mohamed:** Single-cycle architecture, hazard unit logic, bus control design, multicore benchmarking, LRSC hardware implementation.